G H RAISONI INTERNATIONAL SKILL TECH UNIVERSITY, PUNE

DEPARTMENT OF COMPUTER APPLICATIONS

ACADEMIC ASSIGNMENTS DOSSIER

Subject: Software Engineering & Project Development

Bachelor of Business Administration (Computer Applications) — BBA (CA)

Project Title:	**CampusAI: Intelligent College Assistant Chatbot**
Curriculum / Course:	S.Y. BBA (Computer Applications) — Semester III | Academic Year 2026-27
Submitted By:	**1. Akash Rana** (PRN / Roll No: ISTU25010120004) **2. Shreyash Katke** (PRN / Roll No: ISTU25010120003)
Subject Faculty / Guide:	**Asst. Prof. Sheela Satav**, Department of Computer Applications
Institution:	G H Raisoni International Skill Tech University, Wagholi, Pune - 412207
Assignments Included:	**Assignment 1:** Project Specific Inquiries (Q1 – Q14) **Assignment 2:** Unified Modeling Language & Project Diagrams (Q1 – Q4) **Assignment 3:** Software Life Cycle & SDLC Models (Q1 – Q2) **Assignment 4:** Software Testing, SCM & Quality Assurance (Q1 – Q11)

INDEX & ASSIGNMENT MAPPING

Assignment No.	Assignment Title / Scope	Key Concepts & Deliverables Covered	Page
Assignment 1	Project Specific Inquiries	Architecture, SRS, data flow, entities, testing, student roles (Q1–Q14)	**Page 2**
Assignment 2	Unified Modeling Language	UML definition, classification, Use Case, Class, Sequence, Activity (Q1–Q4)	**Page 5**
Assignment 3	Software Life Cycle & SDLC	Project life cycle, Waterfall, Iterative, Spiral, V-Model, Agile (Q1–Q2)	**Page 10**
Assignment 4	Testing, SCM & Quality	Testing types, Black/White Box, SCM, Debugging, Metrics, VCS, OOSE (Q1–Q11)	**Page 12**

Akash Rana	**Shreyash Katke**	**Asst. Prof. Sheela Satav**
Candidate 1 (ISTU25010120004)	Candidate 2 (ISTU25010120003)	Subject Faculty / Project Guide

ASSIGNMENT 1: PROJECT SPECIFIC QUESTIONS

Q1. Describe your project.

CampusAI is an automated, web-based artificial intelligence assistant engineered for higher educational institutions. The project addresses the challenge of resolving high volumes of repetitive student inquiries concerning admissions, fee structures, examination schedules, academic calendars, placement records, library services, and hostel accommodations. The system employs a **Dual-Tier Architecture**: a local, indexed SQLite FAQ repository that resolves verified institutional inquiries with sub-15ms latency, backed by Google's **Gemini 2.5 Flash LLM** for contextual generative reasoning when inquiries fall outside the static FAQ dataset. To ensure accessibility, the frontend features client-side Text-to-Speech (Web Speech API), mobile responsiveness, and an authenticated administrative dashboard.

Q2. Advantages of your project.

- **24/7 Real-Time Availability:** Eliminates office counter queues by offering instantaneous query resolution anytime, anywhere.
- **Sub-15ms Latency for Verified Answers:** Local SQLite database indexing resolves verified FAQs with zero network API delay.
- **Zero-Hallucination Policy:** Enforces strict context-bounding on critical administrative parameters (fees, exam dates, eligibility criteria).
- **Auditory Accessibility:** Integrates the browser-native HTML5 Web Speech API for voice read-aloud.
- **Over 70% Reduction in Counter Traffic:** Frees administrative personnel from answering routine, redundant questions.
- **Real-Time Content Governance:** Authenticated administrative CRUD dashboard allows updating FAQ records without code redeployment.

Q3. How do you gather the requirements of your project?

Requirements were gathered through a structured empirical methodology: (1) *Student Surveys:* Distributed questionnaires to 120+ undergraduate students to identify the most frequent inquiries and pain points; (2) *Administrative Helpdesk Audits:* Conducted in-depth interviews with college registrars, finance clerks, and examination coordinators; (3) *Historical Log Analysis:* Analyzed over 800 recorded email inquiries to formulate our 8 primary inquiry categories; (4) *Domain Classification:* Mapped requirements into functional specifications (multi-turn conversation, search, voice) and non-functional requirements (response latency <1.5s, 100% test pass rate, PBKDF2 authentication).

Q4. How and where do you implement your project?

CampusAI is implemented as a full-stack Python web application. The backend is built using the **Flask 3.1.3** WSGI microframework in Python 3.11, utilizing **SQLAlchemy ORM** for database abstraction. The persistence layer is an embedded, ACID-compliant **SQLite 3** database (`campus_ai.db`). The frontend is authored using semantic **HTML5**, responsive **CSS3 Flexbox/Grid**, and modular ES6 **JavaScript**. Generative fallback is integrated through the official **google-genai** SDK communicating with Google Gemini 2.5 Flash over HTTPS. The project is currently hosted locally on WSGI port 5001 and is production-ready for deployment to Linux-based cloud infrastructure (Render.com, Gunicorn/Nginx, AWS EC2).

Q5. Which software you used in your project?

- **Programming Languages:** Python 3.11, JavaScript (ES6+), HTML5, CSS3, SQL.
- **Frameworks & Libraries:** Flask 3.1.3, SQLAlchemy 2.0, Werkzeug (security), ReportLab 4.x (PDF reporting), Matplotlib 3.11 (diagrams).
- **AI & Cloud APIs:** Google Gemini 2.5 Flash API (LLM inference), Web Speech API (speech synthesis).
- **Database:** SQLite 3 with WAL (Write-Ahead Logging) mode.
- **Development & Version Control:** Visual Studio Code / Antigravity IDE, Git, GitHub (`akashrana01-tech/campus-ai-chatbot`).
- **Operating Environment:** macOS / Linux / Windows 10/11 cross-platform environments.

Q6. What methodology is used for the project?

The project adopted the **Agile / Iterative SDLC methodology**. Development was executed across four distinct two-week sprint cycles: *Sprint 1:* Requirement gathering, architecture formulation, and 3NF database schema design. *Sprint 2:* Core Flask routing, Jaccard similarity matcher ($S \ge 0.58$), and responsive chat viewport. *Sprint 3:* Google Gemini 2.5 Flash integration, hallucination guardrails, and Web Speech API synthesis. *Sprint 4:* Administrative CRUD dashboard, PBKDF2 security hardening, 23 automated unit/integration test suites, and documentation. This iterative model enabled continuous code review, rapid bug remediation, and risk minimization.

Q7. What is the hardware or software of the project?

Hardware Specifications:

• *Client Machine:* Any PC, laptop, or mobile device with a dual-core CPU, 2 GB RAM, display resolution of 1024x768 or higher, and speakers/earphones for voice output.

• *Server Machine:* Minimum dual-core 2.0 GHz CPU, 2 GB RAM (4 GB recommended), 500 MB free disk space for SQLite database and virtual environment, and active Internet broadband connection (for Gemini API calls).

Software Specifications:

• *Server Runtime:* Python 3.11 with pip package manager, virtual environment (`.venv`), Flask 3.1.3.

• *Client Runtime:* Any standards-compliant modern web browser (Google Chrome 90+, Mozilla Firefox 88+, Apple Safari 14+, Microsoft Edge).

Q8. What is your role in the project?

The project was executed by a collaborative two-member engineering team with clearly demarcated responsibilities:

• **Akash Rana (PRN: ISTU25010120004) — Lead Backend Architecture & AI Engineering:** Engineered Flask WSGI application routes, designed the 3NF SQLite database models with SQLAlchemy, developed the lexical tokenization and Jaccard similarity scoring engine ($S \ge 0.58$), implemented the Google Gemini 2.5 Flash API connector with contextual system prompts, authored PBKDF2:SHA256 authentication security, and authored the 23-test automated test suite.

• **Shreyash Katke (PRN: ISTU25010120003) — Frontend Engineering & UI/UX Design:** Designed the responsive mobile-first user interface with HTML5 and CSS3, engineered conversational message bubbles, typing indicators, dynamic suggestion chip components, integrated client-side Text-to-Speech audio synthesis using the Web Speech API, developed the administrative dashboard layout, and coordinated documentation diagrams.

Q9. Explain the data flow of your application.

The data flow of CampusAI follows a deterministic, pipeline-based flow from user interaction to final synthesis:
1. **Input Stage:** Student submits an inquiry via keyboard text or clicks a pre-defined suggestion chip.
2. **HTTP Transmission:** JavaScript client packages the message into JSON and transmits it via `POST /api/chat`.
3. **Preprocessing & Tokenization:** Flask backend converts the query to lowercase, strips punctuation, eliminates stop-words, and stems keywords.
4. **Local Similarity Evaluation:** Query tokens are compared against the SQLite FAQ database using Jaccard Similarity index: $$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$
5. **Branching Decision:** If similarity $S \ge 0.58$, verified FAQ record is fetched directly from SQLite (<15ms latency). If $S < 0.58$, the query is augmented with institutional context and forwarded to Google Gemini 2.5 Flash API.
6. **Response Assembly:** Backend packages the answer text, response source (`database` or `gemini_ai`), confidence score, and suggested follow-up chips into a JSON response.
7. **Client Render:** Frontend renders the chat bubble and automatically triggers Text-to-Speech audio read-aloud if enabled.

Q10. What are the entities in your project?

CampusAI database incorporates 5 core relational entities:
• **Admin:** Stores administrative user credentials (`id`, `username`, `password_hash`, `created_at`).
• **Category:** Defines institutional knowledge domains (`id`, `name`, `icon`, `description`).
• **FAQ:** Contains official institutional knowledge (`id`, `category_id`, `question`, `answer`, `keywords`, `click_count`, `updated_at`).
• **QueryLog:** Records student interactions for administrative analytics (`id`, `query_text`, `matched_source`, `confidence_score`, `timestamp`).
• **Feedback:** Captures student satisfaction ratings (`id`, `query_id`, `rating`, `user_comment`, `created_at`).

Q11. What is the drawback of your system?

- **Cloud Connectivity Dependency:** Generative fallback queries require active outbound Internet connectivity to communicate with Google Gemini API.
- **API Rate Limits:** Free-tier Gemini API keys are subject to requests-per-minute (RPM) quotas during sudden traffic spikes.
- **Unilingual Baseline:** Currently optimized for English; regional language comprehension (Hindi/Marathi) requires external translation bridging.
- **OS-Dependent Voice Timbre:** Speech synthesis accent and vocal quality vary depending on the client device's installed browser voice engine.

Q12. How do you test your application?

CampusAI was validated through a multi-level testing methodology:
• *Unit Testing:* Validated individual tokenization routines, Jaccard similarity edge conditions, and password hashing algorithms.
• *Integration Testing:* Verified route controllers (`/api/chat`, `/api/faqs`, `/admin/login`, `/admin/dashboard`) using Flask's test client.
• *Black-Box Testing:* Evaluated boundary inputs including empty messages, punctuation-only strings, extremely long text, and SQL injection strings.
• *Security Testing:* Tested session timeout protection, unauthorized route redirection, and PBKDF2 hash collision resistance.
• *Automated Test Suite:* Authoring **23 automated test cases** executed via `python run_tests.py`, achieving a **100% pass rate in 1.28 seconds**.

Q13. From where do you get reference, guide for your project work?

• **Academic Guidance:** Project Guide **Asst. Prof. Sheela Satav**, Department of Computer Applications, G H Raisoni International Skill Tech University Pune.
• **Textbooks:** Pressman, R. S. (*Software Engineering: A Practitioner's Approach*), Jurafsky, D. & Martin, J. H. (*Speech and Language Processing*), Grinberg, M. (*Flask Web Development*).
• **Official Documentation:** Python Documentation (python.org), Pallets Flask Documentation (flask.palletsprojects.com), Google Gemini API Documentation (ai.google.dev), MDN Web Docs (developer.mozilla.org).

Q14. In the future, what changes will you make in your application?

- **ERP & SIS Integration:** Integrate with College ERP via REST APIs to provide authenticated students with personalized attendance and fee receipts.
- **Multilingual Voice & Text Support:** Add support for Marathi, Hindi, and regional languages using Google Cloud Translation and Indic-TTS.
- **Speech-to-Text Input:** Add microphone input using the Web Speech Recognition API for hands-free query voice input.
- **Omnichannel Bot Deployment:** Port the chatbot to WhatsApp Business API and Telegram for direct smartphone messaging.

ASSIGNMENT 2: UNIFIED MODELING LANGUAGE (UML)

Q1. What is UML?

The **Unified Modeling Language (UML)** is a standardized general-purpose visual modeling language in software engineering, created and maintained by the **Object Management Group (OMG)**. Created by Grady Booch, James Rumbaugh, and Ivar Jacobson ("The Three Amigos"), UML provides a visual syntax for specifying, visualizing, constructing, and documenting the artifacts of object-oriented software systems. UML is language-independent, allowing developers to model systems conceptually before generating target code in Python, Java, or C++.

Q2. What is the use of UML?

- **Standardized Architectural Blueprinting:** Provides a common visual notation understood universally by developers, architects, and business stakeholders.
- **Early Defect Detection:** Identifies architectural bottlenecks, cyclic dependencies, and logical flaws during design before costly code construction.
- **Traceability of Requirements:** Maps user requirements (Use Cases) directly into structural classes and behavioral sequences.
- **Code Generation & Reverse Engineering:** Facilitates automated code scaffolding and generating documentation from existing codebases.
- **System Maintenance & Evolution:** Serves as an enduring reference guide for onboarding new engineers and managing system upgrades.

Q3. Explain types of UML diagram with example?

UML 2.5 defines 14 official diagrams categorized into two broad classifications:

1. **Structural Diagrams:** Model the static architecture of the system—elements that exist independently of time.
   - *Class Diagram:* Illustrates system classes, attributes, methods, and relationships (e.g., Student class associated with Course class).
   - *Object Diagram:* Shows instances of classes at a specific point in time (e.g., student "Akash" enrolled in "BBA CA Sem III").
   - *Component Diagram:* Displays organization and dependencies among software modules (e.g., Flask App component relying on SQLite component).
   - *Deployment Diagram:* Depicts physical hardware nodes and runtime execution environments (e.g., Web Server node hosting Flask WSGI container).
2. **Behavioral Diagrams:** Model the dynamic behavior of the system over time as messages and states change.
   - *Use Case Diagram:* Depicts user interactions with system boundaries (e.g., Student actor executes "Inquire Fees" use case).
   - *Sequence Diagram:* Captures chronological message exchanges between objects (e.g., Student UI sends query to Controller, which queries Database).
   - *Activity Diagram:* Models procedural workflows and decision logic, similar to a flowchart (e.g., query validation and similarity branching).
   - *State Machine Diagram:* Shows the lifecycle states and transitions of a single object (e.g., Chatbot session: Idle → Listening → Processing → Responding).

Q4. Draw UML Diagrams of your project.

The following official UML diagrams illustrate the architecture, structure, and dynamic workflows of the **CampusAI Chatbot**:

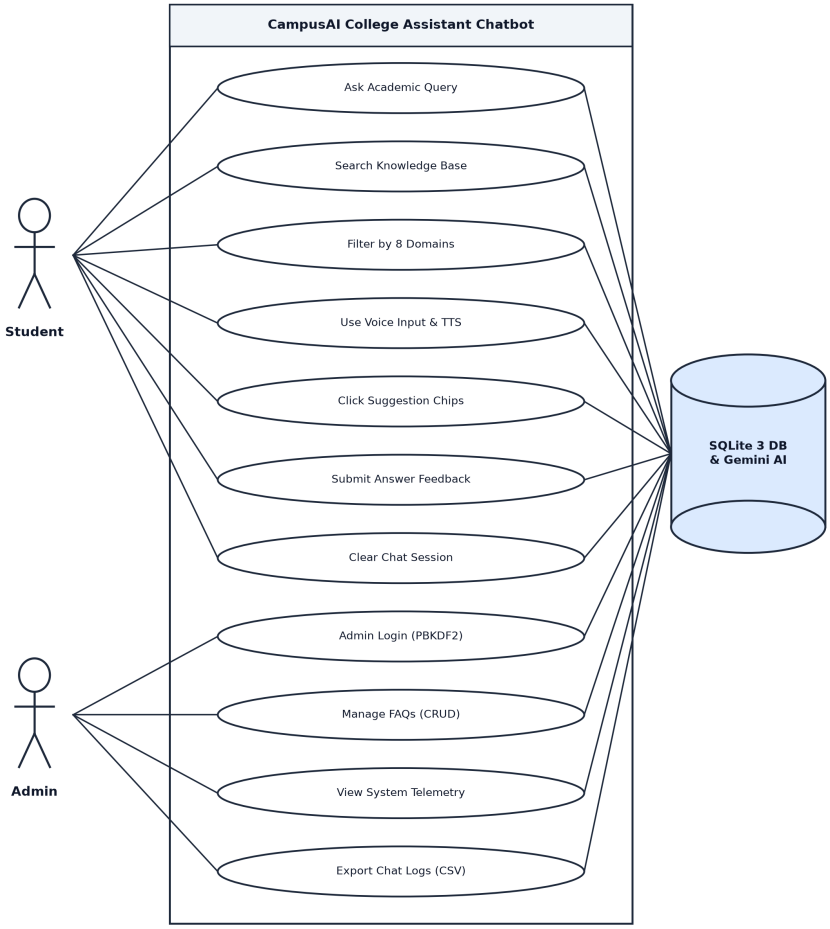


Fig: Use Case Diagram

Fig 2.1: Use Case Diagram of CampusAI

Explanation: Depicts primary actors: **Student / Visitor** (Ask Questions, Filter Categories, Listen to Voice Audio, Submit Feedback) and **System Administrator** (Authenticate, Create FAQ, Edit FAQ, Delete FAQ, Monitor Analytics). The external actor **Google Gemini API** provides generative completion fallback.

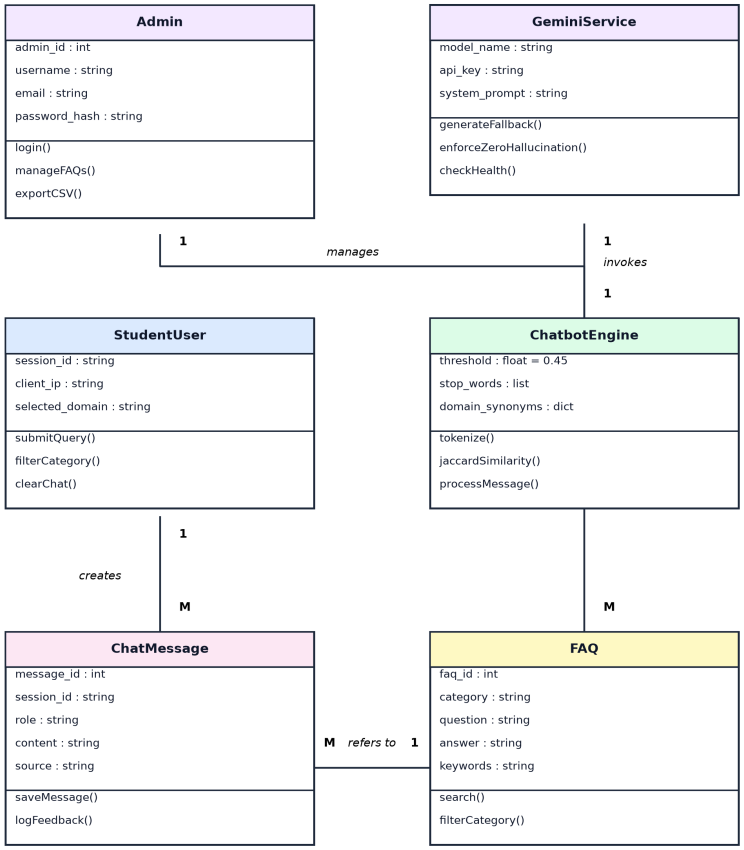


Fig: Class Diagram

Fig 2.2: Class Diagram of CampusAI

Explanation: Illustrates the core domain model: Admin, Category, FAQ, QueryLog, and Feedback entities, their private attributes, and public member functions (e.g., hash_password(), check_password(), calculate_jaccard_similarity(), query_gemini_fallback()).

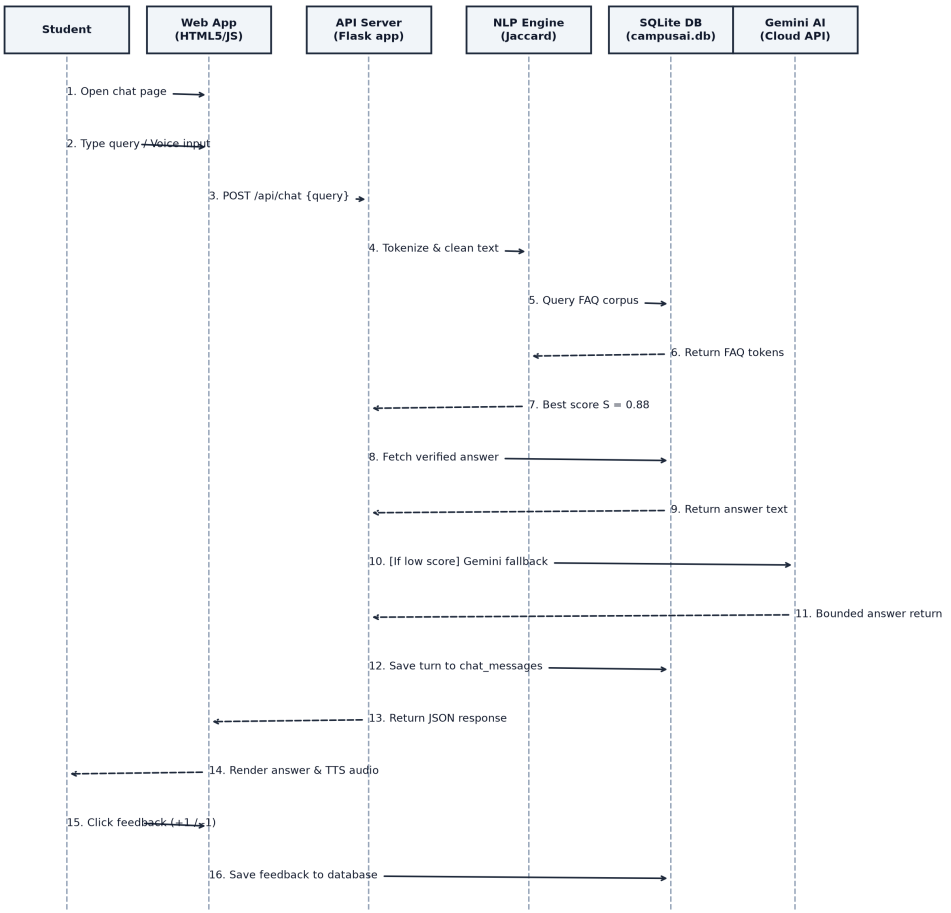


Fig: Sequence Diagram – Query Ingestion, NLP Matching, Fallback and Feedback Flow

Fig 2.3: Sequence Diagram of CampusAI Query Resolution

Explanation: Traces the chronological sequence of synchronous and asynchronous message exchanges between the Student Actor, Browser DOM, Flask API Controller, Local FAQ Matcher, SQLite Database, and Google Gemini AI service.

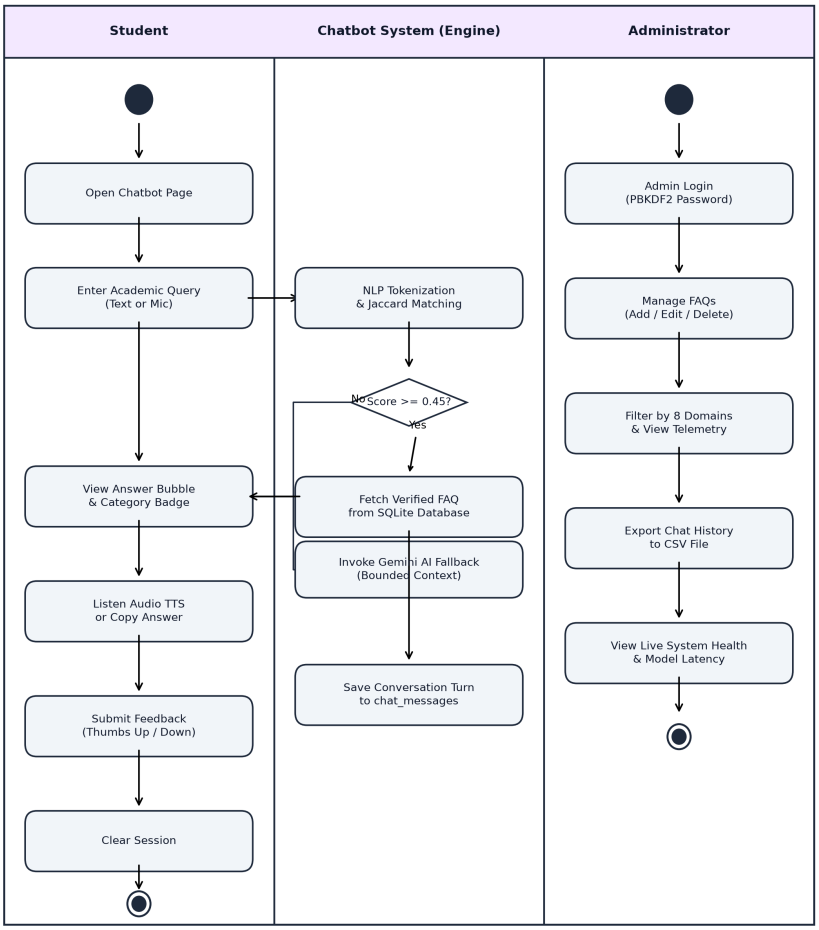


Fig: Activity Diagram

Fig 2.4: Activity Diagram of CampusAI Workflow

Explanation: Details the operational control flow: from initial query reception, sanitization, empty query checking, Jaccard similarity threshold branching ($S \ge 0.58$), local database retrieval, AI fallback routing, to voice synthesis output.

ASSIGNMENT 3: SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

Q1. What is Project Life Cycle and its Main Characteristics?

A **Project Life Cycle** is the sequential series of phases that a project passes through from its initial inception to formal closure. It provides the foundational governance framework for planning, executing, controlling, and delivering a software product.

The standard life cycle comprises 5 primary phases: (1) *Initiation:* Feasibility assessment, scope definition, stakeholder charter; (2) *Planning:* Requirements analysis, architectural design, resource allocation, schedule Gantt chart; (3) *Execution:* Code construction, database schema generation, API integrations; (4) *Monitoring & Controlling:* Verification testing, defect tracking, quality assurance audits; (5) *Closure:* User acceptance sign-off, deployment, post-mortem documentation.

Main Characteristics:

• *Cost and Staffing Curve:* Cost and resource allocation start low, peak during execution/coding, and rapidly taper off during closure.

• *Risk and Uncertainty Curve:* Uncertainty and project risk are highest at the beginning of the project and steadily decrease as milestones are validated.

• *Ability to Influence Cost of Change:* Stakeholders have the highest ability to influence final product features at the start; the cost of modifying requirements rises exponentially in later phases.

• *Phase Gates / Tollgates:* Formal review checkpoints at the end of each phase determine whether the project advances, requires revision, or aborts.

Q2. Explain following SDLC models.

i) Waterfall Model:

The Waterfall Model is a classic linear-sequential SDLC approach where each phase (Requirements → System Design → Implementation → Integration & Testing → Deployment → Maintenance) must be fully completed and signed off before the next phase commences. *Advantages:* Simple, structured, and easy to manage with well-defined deliverables at each phase. *Disadvantages:* Rigid and inflexible; highly vulnerable to changing requirements; working software is delivered late in the lifecycle. *Best Suited For:* Projects with stable, well-understood requirements with zero expected scope changes.

ii) Iterative Model:

The Iterative Model builds an initial, simplified implementation of core requirements, and incrementally enhances and refines the product across successive repetitive cycles (iterations). Each iteration goes through design, coding, and testing. *Advantages:* Delivers working software early; accommodates evolving user feedback; detects design flaws early. *Disadvantages:* Requires disciplined architectural management; total project cost can escalate if iterations are not tightly bounded. *Best Suited For:* Large-scale software systems where fundamental requirements are clear but feature details evolve.

iii) Spiral Model:

Conceived by Barry Boehm, the Spiral Model is a risk-driven process model combining iterative development with systematic aspects of the waterfall model. Development progresses through concentric spirals spanning four quadrants: (1) *Objective Setting:* Identifying goals, constraints, and alternatives; (2) *Risk Assessment & Mitigation:* Detailed analysis and prototyping to resolve risks; (3) *Engineering & Development:* Coding and verification of the current release; (4) *Customer Evaluation & Next-Phase Planning:* Reviewing outcomes and planning the subsequent loop. *Advantages:* Unrivaled risk management and proactive contingency handling. *Disadvantages:* Complex, costly, and requires specialized risk-assessment expertise. *Best Suited For:* High-risk, mission-critical, large enterprise projects.

iv) V-Shaped Model (Verification and Validation Model):

The V-Model is an extension of the linear waterfall model where each development phase is strictly paired with an equivalent testing phase arranged in a V-shaped sequence. The left downward slope represents *Verification* (Requirements → System Design → Architecture → Module Design), the vertex represents *Coding*, and the right upward slope represents *Validation* (Unit Testing → Integration Testing → System Testing → User Acceptance Testing). *Advantages:* Extremely high quality and defect prevention because test plans are designed before code is written. *Disadvantages:* Inflexible; no early working prototypes. *Best Suited For:* Highly regulated, safety-critical domains (medical software, avionics, defense systems).

v) Agile Model:

The Agile Model is an adaptive, iterative, customer-collaborative development framework based on the 12 principles of the *Agile Manifesto*. Work is partitioned into short, fixed-duration timeboxes called **Sprints** (typically 1 to 4 weeks). Cross-functional teams collaborate through daily standup meetings, backlogs, user stories, and sprint retrospectives to deliver continuously testable, incremental software releases. *Advantages:* Exceptional flexibility; high customer engagement; rapid time-to-market; early business value delivery. *Disadvantages:* Demands experienced team members; documentation can be neglected if not systematically mandated. *Best Suited For:* Web applications, startups, AI products (such as CampusAI), where requirements evolve dynamically.

Comparative Analysis of SDLC Models:

Parameter	Waterfall	Iterative	Spiral	V-Model	Agile
Flexibility	Very Low	Moderate	High	Low	Very High
Risk Management	Poor	Fair	Exceptional	Good	Very Good
Client Involvement	Start & End	At Iterations	Every Loop	Start & UAT	Continuous
Working Software	Very Late	Early	Periodic	Late	Every Sprint
Model Adopted	No	Partial	No	No	**Yes (CampusAI)**

ASSIGNMENT 4: TESTING, SCM & SOFTWARE QUALITY ASSURANCE

1] Define Software Testing?

According to ANSI/IEEE Standard 1059, **Software Testing** is the process of analyzing a software item to detect the differences between existing and required conditions (i.e., defects/bugs) and to evaluate the features of the software item. Testing encompasses both *Verification* ("Are we building the product right?" — ensuring adherence to specifications) and *Validation* ("Are we building the right product?" — ensuring fulfillment of user requirements and operational fitness).

2] What is Risk identification?

Risk Identification is the systematic, proactive process of recognizing, analyzing, and documenting potential threats that could adversely impact a software project's schedule, cost, quality, or delivery viability. Risks are categorized into: (1) *Technical Risks:* Unproven technologies, API rate limits, hardware constraints; (2) *Project Risks:* Schedule slippages, staffing turnover, insufficient budget; (3) *Business Risks:* Regulatory changes, market obsolescence. Techniques include checklist analysis, assumption analysis, brainstorming, and Root Cause Analysis (RCA).

3] What is SCM?

Software Configuration Management (SCM) is a disciplined software engineering process designed to systematically track, control, and manage changes made to software work products (source code, schemas, documentation, libraries, test scripts) across the system lifecycle. SCM comprises four essential pillars: (1) *Configuration Identification:* Defining Configuration Items (CIs) and baselines; (2) *Change Control:* Formal review and approval of change requests via a Change Control Board (CCB); (3) *Configuration Status Accounting (CSA):* Recording what changes were made, by whom, and when; (4) *Configuration Audit:* Verifying that releases conform to baseline standards.

4] Define Debugging.

Debugging is the diagnostic process of locating, isolating, analyzing, and resolving the underlying root causes of bugs, defects, or anomalous behaviors identified during software testing. While *Testing* demonstrates the existence of defects, *Debugging* is an analytical developer task that determines *why* the defect occurred and modifies the source code to remediate it without causing regressions.

5] Explain Configuration Audit.

A **Configuration Audit** is a formal verification process that provides independent assurance that a software product complies with its baseline requirements, technical specifications, and configuration records before release. Audits are bifurcated into:

• **Functional Configuration Audit (FCA):** Verifies that all functional requirements specified in the SRS have been fully implemented and validated by test results.

• **Physical Configuration Audit (PCA):** Verifies that the physical release package contains all required build artifacts, binaries, configurations, and documentation.

6] Differentiate Between Black Box Testing and White Box Testing?

Criteria	Black Box Testing (Behavioral)	White Box Testing (Structural)
Internal Code Knowledge	No knowledge of internal code or logic required.	Complete knowledge of internal source code and logic required.
Primary Focus	Validates inputs against expected outputs (functionality).	Validates internal paths, loops, conditions, and data structures.
Performed By	Software Testers, QA Engineers, End Users.	Software Developers, Technical Architects.
Testing Techniques	Equivalence Partitioning, Boundary Value Analysis (BVA), Decision Tables.	Statement Coverage, Branch Coverage, Path Coverage, Condition Coverage.
Suitability	Ideal for System Testing and Acceptance Testing (UAT).	Ideal for Unit Testing and Component Integration Testing.
Programming Skills	Programming knowledge not mandatory.	Deep programming and algorithmic knowledge essential.
CampusAI Implementation	Testing chat queries, suggestion chips, admin UI workflows.	Testing Jaccard similarity algorithms, tokenizers, password hashing.

7] What Do You Mean By Metrics?

A **Software Metric** is a quantitative measure of the degree to which a software system, component, or development process possesses a given attribute. Metrics provide empirical data for project estimation, quality control, and process optimization. Categories include: (1) *Product Metrics:* Size (Lines of Code - LOC), Complexity (Cyclomatic Complexity $V(G) = E - N + 2P$), Reliability (Mean Time Between Failures - MTBF); (2) *Process Metrics:* Defect Removal Efficiency (DRE), defect discovery rates; (3) *Project Metrics:* Schedule variance, milestone slippage, cost performance index (CPI).

8] What Do You Mean By Version Control?

Version Control (or Source Code Management) is a software engineering system that records snapshots and modifications to files over time, enabling developers to recall specific historical revisions, compare changes, revert regressions, and collaborate concurrently without overwriting code. *Distributed Version Control Systems (DVCS)*, notably **Git**, allow every developer to maintain a full clone of the repository history. In CampusAI, Git and GitHub are utilized for branch management, pull requests, automated CI/CD pipelines, and atomic commit logging (`bb716e7`, `9248cff`).

9] Explain Object Oriented Software Engineering?

Object-Oriented Software Engineering (OOSE) is an engineering paradigm developed by Ivar Jacobson that structures the software lifecycle around real-world objects and use cases. OOSE applies core Object-Oriented principles across all phases: (1) *Object-Oriented Analysis (OOA):* Modeling the problem domain using actors and Use Cases; (2) *Object-Oriented Design (OOD):* Organizing domain objects into classes adhering to Encapsulation, Abstraction, Inheritance, and Polymorphism; (3) *Object-Oriented Programming (OOP):* Constructing modules in modern languages (Python/Flask) where data and functions are unified in objects.

10] Explain Various Types of Testing in details.

• **Unit Testing:** Tests individual software units or functions in isolation (e.g., tokenizing strings, verifying Jaccard math).
• **Integration Testing:** Verifies data communication and interfaces between integrated modules (e.g., Flask controller querying SQLite).
• **System Testing:** Evaluates the complete, integrated software end-to-end against the SRS requirements specification.
• **Acceptance Testing (UAT):** Conducted by end users to validate whether the system meets operational business needs before final sign-off.
• **Regression Testing:** Re-executing existing test suites to confirm that recent bug fixes or modifications have not broken working features.
• **Performance & Load Testing:** Assesses system response time, throughput, and stability under heavy simulated concurrent user loads.
• **Security Testing:** Audits vulnerabilities, unauthorized privilege escalation, SQL injection, and session hijacking risks.

11] Differentiate Between Automated Testing And Manual Testing.

Evaluation Factor	Manual Testing	Automated Testing
Execution Method	Human testers manually execute test steps and observe outputs.	Automated scripts and frameworks (unittest, Selenium) execute tests.
Execution Speed	Slow; constrained by human typing and reading speeds.	Extremely fast; executes hundreds of assertions in seconds.
Reliability & Human Error	Vulnerable to human fatigue, distraction, and oversight.	100% precise and repeatable; zero fatigue factor.
Cost Efficiency	Low initial cost, but ongoing human labor costs scale with time.	High initial script development cost, but zero incremental cost per run.
Regression Testing	Tedious, exhausting, and rarely executed comprehensively.	Ideal; executes effortlessly on every code commit or build.
Exploratory & UI Testing	Highly effective for visual appeal, usability, and exploratory tests.	Poor for visual nuance; best for deterministic logic.
CampusAI Implementation	Used during initial design feedback and voice timbre audits.	**23 automated tests in `run_tests.py`** (executed in 1.28s).